

Exploration of Euler's method using Microsoft Excel and Python

Christoffel Schoeman
38655349

22 May 2025

Contents

1	Introduction	3
2	Derivation of Euler's method	3
3	Mathematical Formulas	3
4	Excel Implementation	4
4.1	Newton's Temperature Law	4
4.2	Newton's Second Law	5
5	Python Implementation	5
5.1	Newton's Temperature law	5
5.2	Newton's Second law	6
6	Insight and Reflection	6
7	Lab report investigations	7
8	Application Of Euler's method in other context's	8

1 Introduction

Ordinary differential equations (ODEs) and partial differential equations (PDEs) form a crucial cornerstone of applied mathematics and physics as these mathematical structures describe real world phenomena with great accuracy. However, some differential equations cannot be solved analytically, or are incredibly difficult or time consuming to solve analytically. Thus, approximation methods are employed in order to understand more from the differential equation without having to solve analytically. Many approximation methods exist, and the purpose of this report is to explore Euler's method in this realm.

2 Derivation of Euler's method

Suppose we have a solution to a differential equation. That is, suppose we have some function

$$y = \phi(x) \tag{1}$$

Suppose also that this function has a derivative, expressed as

$$\frac{dy}{dx} = f(y, x) \tag{2}$$

To approximate this solution we first note that the derivative of a function (at a point) can be expressed as a gradient

$$\frac{dy}{dx} = \frac{y - y_o}{x - x_o} \tag{3}$$

This can then also be expressed as

$$y = y_o + \frac{dy}{dx}(x - x_o) \tag{4}$$

This also assumes that the function passes through some point x_o , and the solution to the differential equation has some value

$$\phi(x_o) = y_o$$

3 Mathematical Formulas

Euler's method in the derivation is applied to some general formula, but within this report the focus will be placed upon two formulas. The first of which is Newton's Cooling Law, denoted by:

$$\frac{dT}{dt} = -\lambda(T - T_o) \tag{5}$$

and the second equation being examined is Newton's 2nd law, or two kinematic equations. The first of which calculates displacement, denoted by:

$$x_{n+1} = x_n + v_n dt \quad (6)$$

and the second one is denoted by:

$$v_{n+1} = v_n + a dt \quad (7)$$

Note the following are already within Euler's method's form.

4 Excel Implementation

Firstly, we apply Euler's method, technologically speaking, through Microsoft excel.

4.1 Newton's Temperature Law

Within the Excel folder are a list of constants defined and given associated values. Below that is two columns of data. One containing the changing time values, and another containing the change in temperature values.

Constant	Unit
Lambda	1
t_o	0
T_i	50
Boundary Condition	
delta T	0,01
Time(s)	Temperature(C)
0	50
0,01	49,5
0,02	49,005
0,03	48,51495
0,04	48,0298005
0,05	47,5495025
0,06	47,07400747
0,07	46,6032674
0,08	46,13723472
0,09	45,67586237
0,1	45,21910375
0,11	44,76691271
0,12	44,31924359
0,13	43,87605115
0,14	43,43729064
0,15	43,00291773

4.2 Newton's Second Law

Similarly to the first image, a list of constants is first defined at the top of the spreadsheet. These values are then followed by a series of columns that contain the changing time interval.

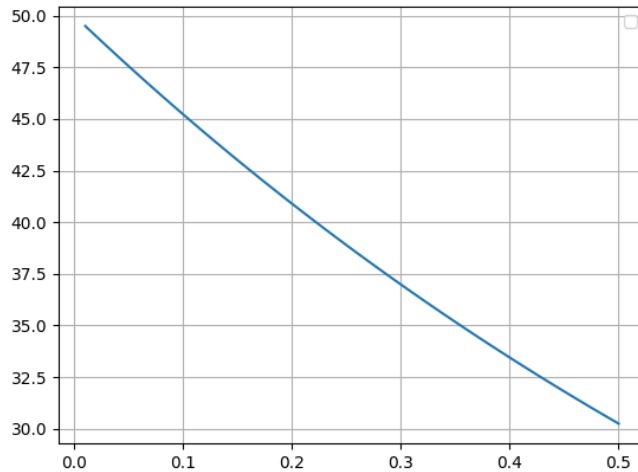
Constant		Unit			
acceleration		1 m/s ²			
m		1 kg			
x_i		0 m			
v_i		1 m/s			
delta t(s)	0.01	s			
BC					
x_f		50 m			
Time(s)	Current Position(m)	Current Velocity(m/s)	Force(N)	Predicted Displacement (m)	Predicted Velocity (m/s)
0	0	1	1	0,01	1,01
0,01	0,01	1,01	1	0,0201	1,02
0,02	0,0201	1,02	1	0,0303	1,03
0,03	0,0303	1,03	1	0,0406	1,04
0,04	0,0406	1,04	1	0,051	1,05
0,05	0,051	1,05	1	0,0615	1,06
0,06	0,0615	1,06	1	0,0721	1,07
0,07	0,0721	1,07	1	0,0828	1,08
0,08	0,0828	1,08	1	0,0936	1,09
0,09	0,0936	1,09	1		

5 Python Implementation

The same procedure that occurred in the Excel portion were also performed in the Python section as well. Two useful libraries were implemented: Numpy and Matplotlib.

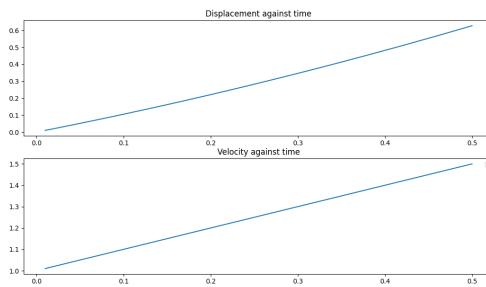
5.1 Newton's Temperature law

Similar to Excel's implementation, the constants are first declared, and then arrays necessary for storing and plotting values are initialised. The formulas are then defined as functions that return scalar values. Then the loop that calculates and stores values are used, and then finally the values are plotted. Attached are the graphs of the temperature law in this context:



5.2 Newton's Second law

Note again that this code, in terms of structure, functions extremely similarly to the temperature law application as the logic behind the code remains relatively the same. Note the graphs presented here.



6 Insight and Reflection

While the excel application did work and print the correct values to that of the python script, it was more tedious to repeatedly implement in the second law equations in excel than in python as the initial position and velocity values had to be manually updated everytime in order for the equation to then be up to date. However in Python, the automation of all parts was quite straightforward and simple to understand. Both printed graphs that yield linear relationships,

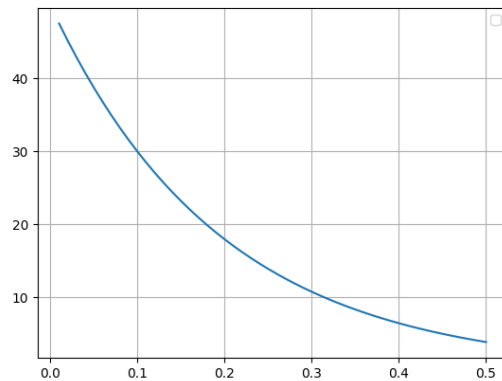
but in Python (with matplotlib) one could customise the graphs a lot more and allow for side by side comparison relatively easily using the subplots code.

As for the application of Euler's method to these equations, it shows quite well that the formula predicts values accurately when compared to the original equations. Suppose now that an incredibly complex ODE was derived. Instead of struggling to solve the equation analytically, one could simply apply Euler's method on some computer programming software to yield accurate results of the formula.

7 Lab report investigations

For the first lab report section, when the code was modified to run the loop until the temperature reached 0, the loop simply never stopped running. Either the function was not defined properly or the temperature cannot reach 0. Mathematically speaking, while the temperature did drop to values incredibly close to 0, it never fully reached 0.

For the second point, changing the lambda value to 5 yields the following result:



An interesting trend is observed. Rather than a linear plot, it is now more curved, or almost hyperbolic in nature. This perhaps imply that higher values of lambda could lead to graphs that match those of different functions or different power values.

For the next point, having a mass of 1kg, the effect of it on both the analytical and numnerical solutions are not of great significance. If one assumes the definition of mass as the resistance to the change in inertia, then a mass of 1kg will (mathematically speaking) not have great impact on the values used here as values will not be scaled at all by 1.

For how the mass effects the time it takes to reach 50 metres, it will take more time to accelerate the body to the same speed as before, and hence take more time to reach 50m.

For the requested derivation, we can derive this from one of the key formulas in kinematics. Namely:

$$x(t) = x_0 + v_0t + \frac{1}{2}a^2 \quad (8)$$

But, from Newton's 2nd law $F_{net} = ma$, we can describe the displacement in terms of force and mass by making acceleration the subject of the equation and substituting that into the displacement formula, which will yield:

$$x(t) = x_0 + v_0t + \frac{1}{2} \left(\frac{F}{m} \right)^2 \quad (9)$$

8 Application Of Euler's method in other context's

Euler's method has proved to be very effective in this context, but it should also be just as useful in other contexts. One notable differential equation is that of population growth in biological fields, denoted by:

$$\frac{dP}{dt} = rP \quad (10)$$

with r being the intrinsic growth rate and P being the population. Now this differential equation is very simply to solve at first, this population does not have any constraints applied to it yet. When things such as food sources, physical space, environments, etc are applied, the equation becomes vastly more complex than before.

Now Euler's method can be applied to these more complex mathematical models to instantly yield results valuable to research or predicting population growth for given populations.

But one problem with this context is that Euler's method only applied to ODE's where the solution function only appears once. If the population requires the description of a 2nd order differential equation or a population that grows at a rate with a higher exponential value. Then Euler's method will not apply so easily.